



MONASH University

FIT3164 Data Science Project 2

Final Project Report

Datalake for the Monash Malaysia Microbiome Research Centre

Group Name: **MDS03**

Team Members:

Chang Lin Ong	(32024509)
Jun Ho Chew	(32409966)
Jia Jing Chai	(31920772)
Tharani Prathaban	(32886047)

Team supervised by:
A/Prof Wong Kok Sheik

Report word count: 10,578 words

***excluding titles, subtitles, diagram descriptions, tables headers, and
any non-paragraph or trivial content

Table of Contents

1.0 Introduction (551 words)	3
2.0 Project Background (1281 words)	5
2.1 Introduction and Project Rationale.....	5
2.2 Literature Review and Context.....	5
2.2.1 Advances in Microbiome Research.....	5
2.2.2 Limitations of Conventional File-Based Storage.....	6
2.2.3 Benefits of Centralised Data Lakes in Life Sciences.....	6
2.2.4 Existing Bioinformatics Tools and Gaps.....	6
2.3 Project Objectives and Significance.....	7
2.4 Improvements over Initial Proposal.....	7
2.5 Deviations and Their Justifications.....	8
2.6 Summary.....	8
3.0 Outcome (4062 words)	9
3.1 What Has Been Implemented.....	9
3.2 Results and Deliverables.....	10
3.2.1 Supported Data Types and Tables.....	10
3.2.2 Final Output and Research Utility.....	11
3.2.3 Key Project Milestones Achieved.....	11
3.2.4 Measured Improvements.....	12
3.3 Evaluation of Project Deliverables Against Specified Requirements.....	12
3.3.1 Requirements Successfully Met by the Project Deliverables.....	12
3.3.2 Partially Met Requirements with Justification.....	13
3.3.3 Omitted Requirements and Reasoning.....	14
3.3.4 Pending Requirements for Future Enhancement.....	14
3.4 Justification of design.....	15
3.5 Project Results – Comparison, Fit, Innovation, Problem-Solving.....	20
3.5.1 Extension of the State of the Art.....	21
3.5.2 Problem Solving and Need Fulfillment.....	21
3.6 Project Limitations & Potential Improvements.....	22
3.6.1 Designed for Only a Specific Set of Files.....	22
3.6.2 Limited Testing with Dynamic, Real-World Data.....	22
3.6.3 Unclear or Technical Error Messaging.....	23
3.6.4 Streamlit Timeout and Upload Stability Untested.....	23
3.6.5 Concurrent Uploads and User Activity Untested.....	24
3.6.6 Limited End-User Autonomy and Customization.....	24
3.6.7 Security and Authentication Handled at Infrastructure Level.....	24
3.6.8 Summary.....	25
4.0 Methodology (2140 words)	26
4.1 Final Project Design.....	26
4.1.1 Architectural Overview.....	26
4.1.2 Detailed Component Design.....	27
4.2 Deviations from Initial Design.....	29

4.2.1 Summary of Initial Proposal.....	29
4.2.2 Analysis of Changes.....	29
4.3 Implementation and Activities.....	31
4.3.1 Development Environment and Toolset.....	31
4.3.2 Implementation Activities.....	32
5.0 Software Deliverables (1126 words).....	33
5.1 Summary of Software Deliverables.....	33
5.1.1 Brief description of deliverables.....	33
5.1.2 Illustrative Screenshots & Use of the Software.....	35
5.2 Software Quality Summary.....	39
5.2.1 Robustness.....	39
5.2.2 Security.....	39
5.2.3 Usability.....	40
5.2.4 Scalability.....	40
5.2.5 Portability.....	40
5.2.6 Maintainability.....	41
6.0 Software and Project Critique (1166 words).....	42
6.1 Overall Project Execution.....	42
6.2 Comparison with Initial Project Proposal.....	42
6.3 Why We Were Able to Stay on Track.....	43
6.4 Challenges Faced and How We Responded.....	44
6.5 Team Collaboration and Project Management.....	45
6.6 What Could Have Been Improved.....	45
6.7 Summary.....	45
7.0 Conclusion (252 words).....	46
8.0 Appendix.....	47
8.1 Updated Requirements Traceability Matrix.....	47
9.0 References.....	55

1.0 Introduction (551 words)

This report documents the design, development, and evaluation of a cloud-based data lake system created for the Monash Malaysia Microbiome Research Centre (MMRC), as part of the FIT3164 Data Science Project Part 2. The system was developed by a team of four final-year data science students and aims to streamline the way MMRC handles, stores, and queries its complex microbiome datasets.

MMRC conducts cutting-edge research on the human microbiome and its link to health and disease. Its research data originates from stool and urine samples and includes bacterial abundance, enzyme and metabolic pathway profiles, and annotated taxonomy, often linked to participant metadata such as age, gender, and lifestyle factors. However, prior to this project, MMRC relied on manual file storage through Google Drive, which presented major limitations: lack of version control, risk of data duplication, poor linkage between datasets, and absence of automated validation, all of which hindered efficient data use and collaboration.

To overcome these issues, this project set out to build a centralized, scalable data infrastructure using Amazon Web Services (AWS). The solution includes capabilities for uploading, validating, transforming, storing, and querying microbiome datasets, with security and data governance embedded throughout. A key goal was to reduce manual handling and ensure data integrity, while enabling researchers to access clean and structured data in real time.

A web-based upload portal was developed using Streamlit, enabling users to submit up to six CSV files, including bacterial abundance, metabolic pathway data, enzyme levels, participant metadata, NMR Asics data, and an optional taxonomy file. Uploaded files are subjected to validation checks, such as duplicate ID detection and completeness of the sample records before being stored in Amazon S3. AWS Glue then processes the files and prepares them for querying through AWS Athena, while user access is governed via pre configured AWS IAM policies.

The student team adopted an agile-inspired workflow, adjusting priorities in response to stakeholder feedback and academic deadlines. Each team member assumed a clear role: Tharani Prathaban (Project Manager), Chew Jun Ho (Data Architect), Ong Chang Lin (Data Engineer), and Chai Jia Jing (Quality Assurance Lead). Regular meetings and collaborative planning ensured alignment and accountability throughout the project.

Key deliverables include the cloud-based S3 folder structure, data ingestion and validation routines, transformation pipelines using Glue, SQL-queryable tables via Athena, and a user-friendly upload interface. Documentation, including a SQL user guide and technical setup guide, was also created alongside to support long-term usability.

While some features, such as data dashboards and downstream analytics integration, were originally planned, they were postponed to prioritize the core infrastructure. These changes were made in alignment with MMRC's immediate needs and long-term goals.

This report is organized to provide a comprehensive account of the project. It begins with the project background, followed by the project outcomes and detailed methodology. The report then presents the software deliverables, and provides a critique of the software and overall project. It concludes with a summary and recommendations for future improvements. Supporting materials such as code listings and references are included in the appendix.

In summary, the project successfully delivers a robust and extensible data management system that addresses MMRC's key pain points. It enhances data quality, accessibility, and governance, while offering the student team valuable experience in full-stack cloud development, data engineering, and collaborative problem solving in a real-world research environment.

2.0 Project Background (1281 words)

2.1 Introduction and Project Rationale

Microbiome science has emerged as a transformative discipline in health research, with studies increasingly revealing how gut microbial communities influence metabolic, immunological, and neurological outcomes across diverse populations. Prior findings have shown that imbalances in microbial populations known as dysbiosis are associated with conditions ranging from obesity, diabetes, and cardiovascular disease to mental health disorders (Ridaura et al., 2013; Wang et al., 2017).

As sequencing technologies, such as next-generation sequencing and mass spectrometry have advanced, microbiome research now generates large and varied datasets. These span from raw sequencing files (e.g., FASTQ format) through to processed abundance tables, taxonomic classifications, and extensive metadata. However, current data management practices at the Monash Malaysia Microbiome Research Centre (MMRC) reveal significant limitations. Files stored on Google Drive frequently suffer from versioning issues, duplication, manual update errors, and poor tracking of dataset lineage, [hindering effective sharing and collaboration](#).

The rationale for this project is to replace this fragmented storage system with a **centralised data lake architecture** that [supports both raw and processed data](#). This centralised platform is designed to improve accessibility, consistency, governance, and scalability, addressing the challenges of working with high-throughput microbiome datasets. By enabling seamless data ingestion, validation, integration, and querying, the data lake empowers multi-disciplinary collaboration and supports advanced bioinformatics workflows essential for uncovering scientific insights.

2.2 Literature Review and Context

2.2.1 Advances in Microbiome Research

The human microbiome's significance has grown substantially over the past decade. Studies by Ridaura et al. (2013) and Wang et al. (2017) have underscored how microbial composition can influence physiological processes and disease development. Dysbiosis, imbalanced microbial communities, has been linked to a broad range of conditions, highlighting the need for robust data handling strategies that support reproducible research and mechanistic exploration.

The advent of high-throughput sequencing and metabolomics has added depth to microbiome studies, yielding extensive raw and processed data streams. These developments demand advanced data architecture; traditional file-sharing methods no longer suffice due to their rigidity and lack of governance.

2.2.2 Limitations of Conventional File-Based Storage

At the MMRC, processed microbiome datasets, often represented as phyloseq objects, are manually curated through high-performance computing (HPC) pipelines. However, this processed data is then transferred to Google Drive, where researchers access and download files for analysis. This approach causes parallel versions to proliferate, creating confusion about which file is current or authoritative. Version control and auditing are nearly impossible, duplicate datasets often accumulate, and collaborative workflows across distinct research teams suffer as a result.

Moreover, integrating datasets across different experimental modalities such as merging microbial abundance tables with sample metadata becomes an error-prone, manual effort. Missing or mismatched sample IDs further compound this issue, increasing the risk of incomplete analyses or inaccurate conclusions (Cernava et al., 2022).

2.2.3 Benefits of Centralised Data Lakes in Life Sciences

Data lakes have emerged as scalable, flexible platforms for storing structured, semi-structured, and unstructured data together. This flexibility offers significant advantages for microbiome researchers:

- **Raw and processed data in one ecosystem:** A data lake supports both FASTQ files and downstream abundance tables, enabling raw data reprocessing if needed.
- **Seamless data governance and versioning:** Systems like Apache Iceberg allow data versioning and even rollback capabilities, enabling reproducibility and traceability over time.
- **Cross-modal data integration:** With unique sample ID linking, researchers can easily join metadata, metabolomic, taxonomic, and abundance data, enabling richer analyses.
- **Enforced validation and quality control:** Automating ingestion and validation reduces manual errors and ensures consistent data quality.

Projects such as **GeniePool** (Hadar et al., 2023) demonstrate how genomic and microbiome datasets can be integrated in cloud-based data lakes. GeniePool efficiently manages diverse data types while maintaining data integrity and provenance. Other reviews emphasize best practices such as clear data governance, scalability, and user accessibility, the core requirements for our proposed system.

2.2.4 Existing Bioinformatics Tools and Gaps

Microbiome analysis tools such as **DADA2** (Callahan et al., 2016), **phyloseq** (McMurdie & Holmes, 2013), and **QIIME** (Caporaso et al., 2010) excel in data processing pipelines: converting raw sequence reads into interpretable taxonomic abundance tables. Yet, they do not provide robust storage, sharing, or version control frameworks. While DADA2 and QIIME manage data cleaning and taxonomy assignment, and phyloseq supports exploratory analysis, none address **centralised storage and retrieval** challenges or ensure consistent versioning across multiple users.

On the storage side, **traditional relational databases** and **data warehouses** impose rigid schema constraints that slow down ingestion and hinder schema evolution, especially problematic in exploratory science. As noted by Kumai & G R (2022), these approaches require upfront schema definitions that do not adapt well to evolving research data types, forcing laborious database redesign when new variables emerge.

In comparison, **data lakes** allow data to be stored "as is" and curated later, better supporting exploratory science. They also enable schema evolution and compatibility with analytical tools such as **AWS Athena**, which can query directly on staging S3 data without requiring upfront schema commitment.

2.3 Project Objectives and Significance

The overarching goal is to develop a secure, centralised, and scalable data lake for microbiome data generated at MMRC. This system will:

1. **Support raw and processed data ingestion** from sequencing and metabolomics pipelines.
2. **Automate structural validation**, detecting missing sample IDs, duplicates across files, and inconsistencies.
3. **Maintain an immutable storage layer**, with a processed and curated layer layered on top.
4. **Enable flexible querying** via [AWS Athena](#) and optionally implement advanced table formats (e.g., [Apache Iceberg](#)) to support INSERT/UPDATE/DELETE operations, enabling data evolution and mutation.
5. **Provide clear data governance**, version control, and data auditing tools to improve reproducibility and collaboration.

Improving the literature review from the original FIT3163/64 proposal, **the new version includes deeper analysis of real-world data lake implementations in biological contexts**, such as Genepool, and underlines the importance of version control, schema evolution, and reproducibility in microbiome science.

2.4 Improvements over Initial Proposal

In the initial FIT3163/64 proposal, the core idea centered on a central data repository with automated uploads. However, the revised approach:

- Incorporated **validation logic** during ingestion to catch data quality issues early.
- Introduced **architectural depth**, including raw and processed storage layers.
- Integrated **interactive querying with Athena**, with future support for Iceberg to allow data updates, which was not part of the original plan.
- Added strong **data governance**, authentication policies, and audit trails, which are the enhancements informed by stakeholder feedback during development.

These enhancements move beyond a passive storage solution toward an active data platform that ensures integrity, supports data evolution, and encourages cross-functional research.

2.5 Deviations and Their Justifications

We intentionally deviated from some initial proposal components based on technical feasibility and developer bandwidth:

- **Skipping real-time streaming ingestion:** Instead of fully streaming raw FASTQ files, we implemented a batch ingestion pipeline due to time constraints and lower operational urgency. The batch process aligns with stakeholder workflows and simplifies validation.
- **Postponing full Iceberg table adoption:** While Iceberg offers powerful capabilities such as ACID transactions and time travel, implementation exceeded our deployment window. Instead, a layered data approach with future-proofing is used to enable future adoption. The added complexity of schema and metadata handling informed this deferment.
- **Manual metadata schema version control:** We manually synchronized parsed metadata schema to reduce system complexity and prioritize core ingestion and querying features.

Our deviations were guided by continuous validation with stakeholders and cost-benefit evaluations. By focusing on core deliverables, we ensured timely project completion without compromising scientific value.

2.6 Summary

In summary, the project attained its core aims: constructing a centralised data lake that dramatically improves ingestion, validation, and query capabilities over legacy Google Drive storage. We advanced the literature review to contextually ground our architectural choices, aligned with proven implementations in life sciences, and continually refined our approach in response to real-world constraints.

Though certain features such as fully automated schema management were deferred, our work delivers a mature and extensible data platform. Documenting deviations from the original proposal and reflecting on their rationale highlights our adaptive and pragmatic development methodology. Future work will expand the platform's capabilities, but the current system provides a solid foundation for ongoing microbiome research across the MMRC.

3.0 Outcome (4062 words)

3.1 What Has Been Implemented

The implementation of our project, a **cloud-native data lake system for the Monash Malaysia Microbiome Research Centre (MMRC)**, comprises two tightly integrated components: a **Streamlit-based front-end interface** and an **event-driven backend cloud infrastructure** hosted on AWS. These two parts work in unison to simplify the ingestion, validation, storage, and querying of highly heterogeneous microbiome and metabolite datasets.

On the **front end**, we developed a user-friendly web application using **Streamlit**, aimed at non-technical research staff. This interface enables researchers to upload raw data files, particularly microbiome, enzyme, pathway, metabolite abundance, and metadata tables in **CSV format**. As ease-of-use and data consistency were critical goals for MMRC, the application incorporates **multi-layered, automated data validation mechanisms** before any data is ingested.

The **validation logic**, abstracted into three layers, ensures high data fidelity:

1. **Intra-file Duplicate Checking:** Upon upload, each file is checked for internal duplicate **sample_id** values. If duplicates are found, they are surfaced on the interface with a preview, and a "Remove Duplicates" button is enabled. In consultation with the researchers, we decided to automate the deduplication by retaining the **first occurrence** and dropping all subsequent duplicates to reduce manual decision-making and streamline workflows.
2. **Cross-file Consistency Validation:** The system then verifies that all required files (e.g., bacterial abundance, enzyme, pathway, NMR ASICS, taxonomy, and participant metadata) share a **consistent and complete set of sample_ids**. Any mismatch is flagged, and incomplete records can be excluded at the user's discretion via a "Remove Incomplete Records" button.
3. **Historical Conflict Resolution:** Finally, the system checks for **sample_ids** that have **already been ingested** in the past. Using **DynamoDB**, a NoSQL document store, we track previously uploaded **sample_ids** by hashing them and storing them in a lookup table. New uploads are checked against this store to prevent redundant insertions into the data lake.

Once all validations are passed, the user can trigger the upload with a single button, after which the files are uploaded to **designated source folders** in the **raw layer of AWS S3**, the cloud storage foundation of the data lake.

From here, the **backend pipeline takes over**, built on a modular, **event-driven serverless architecture**. The S3 bucket is configured to emit an **object creation event** whenever a new file is uploaded. This event is published to **AWS Simple Notification Service (SNS)**, which in turn pushes a message to **AWS Simple Queue Service (SQS)**. SQS acts as a buffer, decoupling upload events from processing jobs and feeding them in sequence to **AWS Lambda**, our serverless compute layer.

Each Lambda function triggers the corresponding **AWS Glue job**, which uses **Apache Spark** as the distributed processing engine to perform the ETL (Extract, Transform, Load) operations. These jobs convert the validated CSV files into **columnar format** and append them into **Apache Iceberg tables**, which reside in the **processed layer of S3**. Iceberg, as a data format, allows for high-performance, transactional inserts and metadata-aware queries, which is ideal for batch-wise, append-only scientific datasets.

The **final query layer** is implemented using **AWS Athena**, a serverless interactive SQL engine, enabling researchers to explore the processed data using standard SQL. Athena reads directly from the Iceberg tables and can return results rapidly due to the columnar storage and indexing structure.

To support the researchers further, we provided a **deployment-ready version of the web interface on Streamlit's Community Cloud** with private access settings to avoid search engine indexing. For handover, we documented procedures for MMRC to host the same app using their own AWS and Streamlit accounts. We also included a **custom SQL query guide** for Athena, along with usage examples and troubleshooting notes, ensuring the system is usable and maintainable by research staff without a software background.

3.2 Results and Deliverables

The implementation phase culminated in the successful delivery of a **fully functional, end-to-end data ingestion and query system** tailored to MMRC's research needs. The platform is now capable of ingesting, validating, transforming, and making accessible large volumes of batch-wise microbiome and metabolite data across multiple experimental pipelines.

3.2.1 Supported Data Types and Tables

The system currently ingests **CSV files** for the following data categories:

- **Bacterial abundance**
- **Enzyme abundance**
- **Bacterial pathway activity**
- **Urine metabolite abundance via NMR ASICS**
- **Participant metadata**
- **Taxonomy reference tables** (optional but occasionally needed for master table's reference updates when newly sampled data has readings recorded for never-seen-before bacterias)

These datasets are processed and stored in the following **Apache Iceberg tables**:

mmrc_bacterial_abundance_sam	mmrc_bacterial_abundance_tax
mmrc_bacterial_pathway_sam	mmrc_bacterial_pathway_tax

mmrc_enzyme_abundance_sam	mmrc_enzyme_abundance_tax
mmrc_nmr_asics_abundance_sam	mmrc_nmr_asics_abundance_tax
mmrc_participant_metadata	
mmrc_taxonomy_table	

Each of the 4 tables, namely **bacterial_abundance**, **bacterial_pathway**, **enzyme_abundance** and **nmr_asics_abundance**, is created in **two formats** to accommodate researcher preferences; one in wide format with **sample_id** as the row index and feature variables as columns, and another in long (transposed) format to support more dynamic queries involving both **sample_id** and specific **taxonomy_id** associated with each of the feature variables (bacterias) in the former table. This dual representation ensures flexible querying, particularly for correlation and joined attributes analysis across datasets.

3.2.2 Final Output and Research Utility

The final deliverable is a **centralized, searchable, and expandable data lake** that enables MMRC to:

- Maintain a **unified source of truth** for all sampled microbiome data and batch uploads
- Query across historical and newly uploaded data instantly via Athena
- Validate and track the quality of their data at each ingestion point
- Extend the system with additional data formats or taxonomic hierarchies in the future

Researchers now have a single, consistent, and secure location from which they can query pre-processed microbiome data, reducing reliance on fragmented spreadsheets and ad hoc scripts. Additionally, the taxonomy table can be periodically updated by researchers by uploading new taxonomic references, which then propagate downstream during query processing.

3.2.3 Key Project Milestones Achieved

1. **Front-end application completed** with multi-layer validation and dynamic file management
2. **S3 raw layer ingestion and folder structure** established
3. **Serverless architecture (SNS → SQS → Lambda → Glue)** implemented and tested
4. **Glue jobs developed for each file type**, supporting Apache Iceberg write compatibility
5. **Dynamic, on-the-fly schema design and Iceberg table creation** finalized with transposed and non-transposed formats
6. **Athena querying capability enabled** with prepared sample queries and error-handling documentation

7. **Private deployment of Streamlit app** for testing and internal use
8. **Handover plan and researcher-facing documentation** completed, including code reuse instructions

3.2.4 Measured Improvements

The most tangible improvement is the **elimination of fragmented storage and manual ingestion workflows**. Previously, researchers were storing datasets locally or on ad hoc drives and had to rely on memory or informal tracking methods for ingestion history. With the new system:

- **All data is centralized on S3**, with associated upload metadata for version tracking
- **Duplicate and incomplete record detection is automated**, saving hours of manual QC
- **Moderate-performance query access is enabled via Athena**, reducing the barrier to exploratory analysis
- **Data corruption and job failure feedback** is automatically surfaced through AWS Glue job logs, providing early warnings to avoid propagating bad data

In short, the current platform creates a **strong, scalable foundation** for MMRC's microbiome research data, streamlining not just ingestion but also the **consumption and collaboration** around it.

3.3 Evaluation of Project Deliverables Against Specified Requirements

3.3.1 Requirements Successfully Met by the Project Deliverables

Several high-priority requirements were directly implemented and satisfied through our system architecture and development work. For example:

1. Requirement 003 – Data Validation:

Our deliverable features a **3-layer validation system** built into the Streamlit front-end that checks for schema conformity, missing or duplicate IDs, data type mismatches, and overall completeness before files are ingested into AWS. This ensures high data quality and integrity, fulfilling this critical non-functional requirement.

2. Requirement 006 – Batch Processing of Datasets:

The ingestion UI supports grouped file uploads, and Glue scripts handle **batch-wise processing**. This allows researchers to upload multiple files per session and process them concurrently, aligning with real research practices involving batch data exports.

3. Requirement 007 – Storage in Data Lake:

We designed and deployed a system that stores both **raw microbiome files in S3** and **processed, structured data as Apache Iceberg tables**. This fulfills the

requirement for centralized storage that supports querying, scalability, and evolution of the dataset.

4. Requirement 008 – Encryption in Transit and at Rest:

Our use of AWS services (S3, Athena, Glue) automatically meets this requirement. AWS encrypts data at rest using server-side encryption (SSE-S3) and encrypts in transit via HTTPS, ensuring data security with no added configuration burden.

5. Requirement 010 – Linking to Personal Identifiers:

The ingestion logic allows uploading **participant metadata** separately and supports structured querying by joining this metadata with microbiome or metabolomics data, ensuring that **samples can be tied to personal attributes** such as age, gender, or state.

6. Requirement 012 – Query Support:

The system enables researchers to run **custom SQL queries via Athena**. A query guide is provided to help them extract insights relevant to their study, such as the presence of a particular bacterial species in specific demographic subgroups.

3.3.2 Partially Met Requirements with Justification

Some requirements were partially fulfilled due to external constraints or future planning needs:

1. Requirement 014 – Role-Based Access Control:

AWS IAM permissions are in place for backend access, but **researcher IAM roles** are not yet set up. Currently, the Streamlit UI is private and meant for internal use. Full implementation will require collaboration with the research team post-handover.

2. Requirement 018 – Uptime Guarantee:

The backend leverages AWS infrastructure, which ensures strong uptime. However, the **Streamlit Community Cloud** frontend lacks formal uptime SLAs. This makes it suitable for testing and research use, but not yet production-grade.

3. Requirement 019 – Query Latency Under 5 Seconds:

For most queries and moderate data volumes, Athena's performance is acceptable. However, **initial query execution** (particularly cold-starts) can experience a delay due to metadata scanning in Iceberg tables.

4. Requirement 020 – Logging Access and Modifications:

AWS provides basic logging (e.g., S3 access logs and Glue job histories), but **CloudTrail-level audit logging** is not yet configured. This can be added later to meet compliance or auditing standards.

5. Requirement 021 – Data Recovery Mechanism:

S3 versioning and durable storage offer baseline recovery, but explicit **roll-back workflows** or recovery automation for corrupted uploads are not yet formalized.

3.3.3 Omitted Requirements and Reasoning

Some originally defined requirements were omitted during development due to evolving project focus or feasibility:

1. **Requirement 001 – Ingest from Google Drive:**

This was replaced with direct ingestion via the Streamlit UI. Google Drive integration was excluded due to its added complexity and lack of scalability. The current method is more secure and controllable.

2. **Requirement 002 – Automated Pipeline Ingestion:**

Direct ingestion from lab pipelines was not feasible without deeper integration into MMRC's HPC environment. Instead, our system accepts **manual uploads of processed CSVs**, providing a flexible middle ground.

3.3.4 Pending Requirements for Future Enhancement

Certain features are acknowledged as important but are left for future development:

1. **Requirement 011 – Metadata Versioning:**

Currently, new metadata uploads overwrite older entries. A **version-controlled metadata history** is not in place yet but can be integrated into Iceberg in future iterations.

2. **Requirement 016 & 017 – Visualization Integration and Dashboards:**

The current Streamlit UI handles ingestion and basic feedback but does not support custom dashboards. However, **Athena supports direct integration with data visualisation tools like Amazon Quicksight**, and this could be added easily later.

3.4 Justification of design

Component	Design Choice	Justification / Reasoning
Front-End Framework	Streamlit	Allows fast development using Python , which aligns directly with the team's and researchers' skills. Alternatives like React or Angular were not considered due to their steep learning curve and higher deployment complexity. User-friendly UI tailored for non-technical users was critical for the researcher audience.
Deployment of Front-End	Streamlit Community Cloud (Private)	Avoids server/infrastructure management overhead. Commercial cloud providers or custom-hosted solutions were not considered due to internal scope. Provides basic email restriction access , ensuring minimal but sufficient security. Planned upgrade to Monash SSO for future stronger authentication.
Upload Method	Manual file uploads (CSV format) via UI	Chosen for simplicity and researcher control. Automation pipelines were avoided to prevent ingestion of unvalidated or erroneous data . Manual review before upload reduces the risk of corrupted/incomplete data entering the system. Automation was not considered due to complexity and risk early in the development cycle.
Ingested File Format	CSV	Selected because CSV is a universal, simple export format common in labs , making it accessible and easy to validate and parse. More complex formats (e.g., JSON, Parquet) were not considered due to researcher familiarity and the extra complexity in validation and tooling.
Data Validation Layer	3-tier validation in Streamlit	Implementing validation on the client side ensures that data issues (e.g., schema mismatches, missing IDs, or duplicates) are caught before the data reaches the backend. This reduces unnecessary storage and compute costs and provides instant feedback to users. Server-side validation was considered , but deemed overly complex and inefficient for this early ingestion stage. The front-end approach also aligns with Streamlit's real-time UI responsiveness.

Intra-file Duplicate Handling Mechanism	First occurrence retained, rest auto-removed via single click	Allowing users to manually resolve duplicates was intentionally avoided to prevent decision fatigue, especially when large datasets contain many duplicate entries. Instead, the system is designed to automatically retain the first valid occurrence of each record and remove the rest with a single user action. This simplifies the workflow while still maintaining a level of user control and data integrity.
Reference Database for Validation	DynamoDB	Chosen because traditional relational databases would require scanning entire datasets for duplicates, which is inefficient. DynamoDB's NoSQL key-value structure enables fast, scalable, real-time duplicate checks without performance bottlenecks. Alternative stores like RDS or Elasticsearch were not chosen due to cost and complexity for this specific function.
File Storage (Raw Data)	Amazon S3 (organized into raw and processed folders)	S3 was selected for its durability, scalability, and integration with AWS analytics tools. Other cloud storage options (Azure Blob, Google Cloud Storage) were not used to maintain a consistent AWS ecosystem. Separating raw and processed data ensures clear lineage and auditability , which is critical for reproducibility and data governance.
Event Trigger Mechanism	S3 → SNS → SQS → Lambda	This asynchronous, event-driven pipeline decouples ingestion from processing , allowing scalability and modularity. A synchronous or monolithic approach was avoided to prevent bottlenecks and improve fault tolerance . The choice also reflects AWS best practices, making future enhancements and maintenance easier.
Processing Engine	AWS Glue + Apache Spark	Spark was selected for its powerful distributed processing capabilities essential for large microbiome/metabolomics datasets . Glue abstracts infrastructure management, reducing operational overhead. Other processing frameworks like EMR or custom clusters were not selected due to increased cost and management complexity.

Transformation Layer Output	Apache Iceberg Tables on S3	Iceberg was chosen to support schema evolution, ACID transactions, and time-travel features , which traditional Glue tables (with Parquet files) cannot provide. This enables rollback and auditability, which are critical for research data integrity. Alternatives without these capabilities were not acceptable given the project's requirements.
Query Layer	AWS Athena	Athena enables serverless, cost-efficient SQL queries directly on Iceberg tables, requiring no infrastructure management. Other query engines (Presto, Redshift) were not considered due to additional complexity and cost. Athena also supports integration with QuickSight , an AWS Data Visualisation tool, for future visualization needs.
Time-Travel Support	Apache Iceberg Snapshots	Time travel enables viewing or rolling back to prior dataset versions, reducing the need for separate backup systems . Traditional backup snapshots were not implemented as Iceberg's built-in versioning sufficiently meets rollback requirements with less complexity.
Metadata & Participant Data	Stored as separate Iceberg tables	Separating participant metadata from microbiome/metabolomics data was chosen to simplify data management, updating, and querying. Keeping them together was avoided due to complexity and to support flexible join operations . This separation also helps maintain privacy controls more easily.
Access Control	AWS IAM (partially configured)	IAM controls are ready for backend services, but full role-based researcher access is pending Monash IT's involvement. This partial implementation reflects dependency on institutional security policies , and a full solution awaits integration with Monash's identity management system .
Front-End Access Control	Private Streamlit URL with email restrictions	Email restrictions provide basic access control without needing complex authentication setups early on. More secure SSO integration with Monash Okta is planned but not implemented yet due to project timing and dependencies on Monash IT cooperation.

Encryption	S3-managed encryption at rest + HTTPS in transit	Fully managed AWS encryption was adopted to meet security requirements with minimal operational overhead. Custom encryption solutions were avoided to reduce complexity and risk of misconfiguration. This standard approach ensures compliance with institutional and regulatory requirements .
Logging & Monitoring	S3 access logs + AWS Glue job history	Basic logs are implemented for monitoring data access and ETL job executions stored on AWS CloudWatch. More detailed auditing like AWS CloudTrail is not configured yet due to project scope and timeline but can be added as security needs grow.
Recovery Mechanism	S3 versioning + Iceberg snapshot rollback	Recovery relies on built-in versioning features of S3 and Iceberg snapshots. No automated recovery workflows are in place yet, reflecting the project's incremental development approach and prioritization of core functionality first.
User Query Support	Athena + SQL guide	SQL querying was prioritized for researcher flexibility, with a guide developed to aid users unfamiliar with SQL . Developing full GUI-based query builders or visual query tools was not considered at this stage due to resource constraints and to keep focus on backend robustness.
Visualization	Deferred (Streamlit UI minimal; future Tableau/PowerBI)	Visualization is deferred because the primary goal was to build a solid data ingestion and query backend first . Streamlit UI currently offers minimal interaction; richer visualizations will be handled in specialized BI tools later , which provide better features and scalability for dashboards.
Metadata Versioning	Overwrite currently; Iceberg snapshots available	S3 bucket versioning is currently not implemented due to complexity and lower priority. Iceberg's time travel feature offers a fallback mechanism , which was deemed sufficient initially. Full version control can be developed later as requirements evolve.

Batch Processing Support	Multiple file uploads per session + Glue batch jobs	Batch uploads reflect real-world lab workflows where data arrives in groups. Glue jobs handle concurrency efficiently. Real-time streaming was not considered necessary due to the batch-oriented nature of research data acquisition .
Component Modularity	Decoupled front-end and backend	Decoupling was intentional to allow independent development, scaling, and maintenance of UI and backend components. Monolithic approaches were avoided to improve reliability, flexibility, and enable gradual upgrades without impacting the entire system.
Documentation Support	SQL walkthrough and troubleshooting guide	Documentation was prioritized to lower the barrier for researcher adoption given the varied technical backgrounds. More comprehensive training or user support was planned but limited initially to fit project resources and timeline.
Data Layer Separation	Raw and Processed data folders on S3	This separation was chosen to enforce good data governance and auditability practices . Storing all data mixed was rejected to avoid confusion and ensure traceability from raw to processed stages.
Testing and CI/CD	Auto-deploy via Streamlit community cloud on code push	Streamlit's auto-deploy on GitHub pushes was leveraged as a lightweight CI/CD solution due to the project's scope. More robust CI/CD pipelines and backend deployment automation were not implemented to save time and reduce operational overhead during initial development phases.
Concurrency Limits	Upload limited to 1 user at a time; query open to many	Upload concurrency is intentionally limited due to lack of stress testing and to control complexity during development. Query concurrency is supported by Athena, which was leveraged to enable multiple simultaneous data analyses . More advanced upload concurrency handling is planned for future iterations.

3.5 Project Results – Comparison, Fit, Innovation, Problem-Solving

Our project deliverables present a substantial improvement over the previous data handling methods employed by the Monash Microbiome Research Centre (MMRC). Prior to our solution, researchers at MMRC relied on manually curated and processed microbiome datasets generated through HPC pipelines. These datasets, often in the form of `.csv` exports from `phyloseq` R objects, were stored and shared via Google Drive. This led to disorganized data storage, lack of version control, duplication of datasets, and significant confusion in identifying the authoritative versions for analysis. Moreover, these scattered datasets impeded collaboration across research teams and made auditing or automated tracking of data evolution nearly impossible.

In contrast, our solution introduces a structured, centralized, and semi-automated ingestion pipeline backed by scalable AWS cloud infrastructure. The pipeline processes six core files (metadata, bacterial abundance, bacterial pathway, enzyme abundance, metabolite abundance, and ASICS metabolite table), validating them through a deterministic three-step check for schema completeness, data types, and missing values. This guarantees higher data integrity and consistency across ingested records.

Compared to the previous manual upload method, our ingestion system offers structured validation, version-awareness (even though not full-fledged version control), and automated routing of data into the data lake. No ingestion tools were previously in place; our solution eliminates error-prone, manual curation while enabling more systematic archival of research data. We also introduce new tools for researchers to interact with their datasets through a user-friendly UI built with Streamlit and backed by a SQL-based querying layer, removing the need to open or cross-reference spreadsheets manually.

On the performance front, validation processes are deterministic and transparent, where every ingested row is checked against expected types and structures. The query performance initially experiences latency, particularly during the first query execution. This is expected due to table loading and cold starts associated with the underlying AWS Athena and Apache Iceberg setup, which build and cache metadata indexes upon initial access. However, subsequent queries improve substantially in speed, thanks to Athena's caching and Iceberg's partition-aware architecture.

Our deliverables strongly align with the needs of microbiome researchers. In particular, the following pain points are addressed:

- **File organization:** All ingested files are organized by upload datetime, with unique identifiers and S3 event-triggered metadata registration.
- **Schema flexibility:** The system allows ingestion of files with additional columns, supporting the evolving nature of experimental data.
- **Data validation:** The solution ensures that incomplete or malformed records are flagged or rejected before ingestion.
- **Centralization:** A unified database of validated microbiome data aids in collaborative research, tracking, and reproducibility.

- **Usability:** Researchers can now interact with data through an interface that hides technical complexities, such as S3 storage structures or AWS Glue logs.

We received iterative feedback from the MMRC team during development. One notable outcome from these discussions was the abstraction of S3 access through a simple upload interface, which was developed after we realized that giving direct access to S3 could overwhelm non-technical users. This feedback loop ensured alignment with their actual workflows, such as requiring queries on specific bacteria across sample age ranges or filtering metabolite data by gender.

3.5.1 Extension of the State of the Art

This project introduces multiple architectural innovations that MMRC had not previously employed:

- **Event-driven ingestion** via S3 triggers and AWS Lambda.
- **Automated duplicate detection** based on primary keys and file-specific logic.
- **Support for schema evolution** through Apache Iceberg tables and metadata decoupling.
- **Time-travel capabilities** offered by Apache Iceberg, enabling the system to query or revert to previous states of the data lake at specific points in time.
- **Data abstraction layer** that supports query composability across unrelated files.

These components make our system scalable and maintainable. Unlike their previous setup, which required file-by-file management, our solution enables ingestion of hundreds of samples without modifying core infrastructure. Although the current system does not allow deletion of data, we built the foundation for a super-user control layer to be added in future work. Moreover, the source of truth is no longer tied to the raw files, but to dynamic SQL queries composed over properly joined and validated tables.

We believe this solution is generalizable beyond MMRC. Any biomedical or environmental research lab dealing with high-throughput, batch-based, and evolving datasets, especially ones using R-based processing pipelines would benefit from the centralized validation, ingestion, and querying capabilities our tool offers.

3.5.2 Problem Solving and Need Fulfillment

The solution solves several critical issues faced by MMRC:

- Eliminates **file scattering** and inconsistent naming/versioning.
- Replaces **manual validation** with a deterministic ingestion protocol.
- Automates the **tracking of ingested datasets** and their metadata.
- Reduces reliance on **spreadsheets and ad-hoc scripting** for data exploration.
- Facilitates **collaborative access** and removes ambiguity around which dataset version to use.

Additionally, researchers are no longer burdened with checking for NA values, missing headers, or malformed data types. Our preprocessing pipeline and SQL access layer

empower even non-technical users to filter, aggregate, and analyze data from multiple studies without direct scripting.

By aligning our architecture to support future expansion (e.g., additional modalities such as urine metabolomics), this system marks a foundational step towards sustainable, research-oriented data infrastructure at MMRC.

3.6 Project Limitations & Potential Improvements

While our solution successfully addresses key data management needs for the Monash Microbiome Research Centre (MMRC), several limitations were identified during development and testing. These constraints arose due to the project's finite timeline, the evolving nature of microbiome research data, and the complexities of cloud-based data infrastructure. This section outlines and explains these limitations, their underlying causes, potential impacts, and opportunities for future improvement.

3.6.1 Designed for Only a Specific Set of Files

Limitation: The current system was designed to ingest and validate a predefined set of six datasets, namely, bacterial abundance, bacterial pathway, enzyme table, metadata, NMR (ASICS) metabolite abundance, and a reference dictionary. These were carefully selected based on domain knowledge and stakeholder input at the beginning of the project.

Reason/Context: Microbiome datasets are inherently complex and diverse. Each data table can follow its own structural conventions and semantic interpretations. Due to this diversity, we had to explicitly define the ingestion logic, validation rules, and transformation steps for each file type individually.

Impact: As a result, the pipeline is not easily extendable to ingest *new* data types or alternative formats (e.g., time-series data, genomics, metatranscriptomics) without significant manual intervention. Any attempt to ingest unfamiliar files would require in-depth knowledge of the domain and the new data structure, potentially leading to system instability or failure if misconfigured.

Proposed Improvement: Future development should consider a **plug-and-play ingestion model**, where researchers can define new data types through configuration files or a metadata schema registry. This would reduce developer dependence and make the system more scalable and resilient to future research needs.

3.6.2 Limited Testing with Dynamic, Real-World Data

Limitation: During the development phase, the data pipeline and front-end interface were tested using either static mock datasets or limited real datasets available at the time. While these allowed us to verify core functionality, they did not capture the full variability found in real-world experimental data.

Reason/Context: Real microbiome data, especially from stool samples or metabolomics studies, is prone to frequent changes with new columns, variable sample sizes, changes in naming conventions, and the addition of unforeseen data fields. The pipeline was not stress-tested against such evolving data landscapes.

Impact: The lack of dynamic testing limits the robustness of the solution in production settings. Unexpected changes in future datasets (e.g., renamed columns or missing expected fields) may cause ingestion failures or corrupt downstream analysis.

Proposed Improvement: Introduce **adaptive schema validation**, where researchers can define flexible schema rules or use profile-based schema inference. Include periodic ingestion tests using real-world data batches to ensure continued reliability as the research evolves.

3.6.3 Unclear or Technical Error Messaging

Limitation: When Glue jobs failed, either due to validation errors, missing dependencies, or schema mismatches, the notifications and logs provided were ambiguous and highly technical, targeted at developers rather than researchers.

Reason/Context: AWS Glue, by default, provides logs through CloudWatch, which are verbose and difficult for non-technical users to understand. In some cases, errors such as “Error Category: UNCLASSIFIED_ERROR; IndexError: StructType index out of range” could leave non-technical users like the researchers confused about what went wrong.

Impact: Researchers without a technical background were unable to debug ingestion issues, diagnose failed uploads, or correct formatting problems, which hinders independent usage of the platform.

Proposed Improvement: Develop a **user-centric error reporting layer**, where common errors are parsed and rephrased into actionable messages. In addition, include a troubleshooting guide within the UI that helps users correct errors without needing developer assistance.

3.6.4 Streamlit Timeout and Upload Stability Untested

Limitation: The front-end interface, built using Streamlit, was primarily tested with small- to medium-sized CSV files. The behavior of the system during large uploads or in slow network environments was not thoroughly evaluated.

Reason/Context: Local testing and limited staging environments made it difficult to simulate long upload durations or network interruptions. Streamlit does not natively handle background uploads or partial transfer recovery.

Impact: Uploading large files (especially >200MB) may lead to front-end freezing, timeouts, or even corrupt uploads, which can frustrate users and lead to data loss.

Proposed Improvement: Integrate **asynchronous upload handling** with progress bars, server-side session tracking, and retry mechanisms. Alternatively, switch to a more scalable web framework for this feature or enable pre-signed S3 uploads directly from the browser.

3.6.5 Concurrent Uploads and User Activity Untested

Limitation: The system was designed and tested assuming a single-user upload model. Scenarios where multiple researchers concurrently upload different files or attempt to modify the same dataset were not explored.

Reason/Context: Stress-testing for concurrency requires sophisticated load testing tools and multi-user environments, which were beyond the scope of this phase.

Impact: Without proper concurrency control, simultaneous uploads may result in data collisions, overwrite errors, or ingestion of duplicate records. These conditions pose a risk to data integrity and consistency.

Proposed Improvement: Simulate real-world usage using tools like **Locust** or **JMeter** to identify bottlenecks and implement locking mechanisms, file-level checks, or versioning to handle concurrency gracefully.

3.6.6 Limited End-User Autonomy and Customization

Limitation: While the system provides researchers with a simplified upload interface and querying mechanism, it still relies on developers for new query logic, ingestion updates, and schema adjustments.

Reason/Context: The backend structure (Athena, Glue, Iceberg) is inherently technical and requires domain understanding to configure properly. We opted for hard coded logic to ensure data integrity and limit user-induced errors.

Impact: This reduces flexibility for power users who might want to define their own queries, include new sample metadata, or link external datasets to the lake.

Proposed Improvement: In future iterations, introduce a **query builder tool**, allow upload of SQL templates, or include a metadata catalog UI that lets researchers browse schema and construct queries visually.

3.6.7 Security and Authentication Handled at Infrastructure Level

Limitation: While access to S3, Athena, and the UI is secured using IAM roles and limited credentials, the user-facing layer does not yet implement fine-grained role-based access control (RBAC) or audit logs at the interface level.

Reason/Context: IAM policies provided sufficient protection for early-stage testing. However, with more users or external collaborators, user-specific access controls will become essential.

Impact: Lack of RBAC could expose sensitive data to unintended users or prevent granular access management (e.g., “Read-only” access for external auditors).

Proposed Improvement: Implement an **RBAC layer in the UI** (e.g., Streamlit with login, session tokens) and integrate audit trails for all user actions like upload, delete, or query execution.

3.6.8 Summary

While the current system presents a significant improvement over previous workflows, these limitations highlight areas for future enhancement. By addressing these issues, particularly around scalability, error handling, testing with live data, and user autonomy, the platform can evolve into a fully production-ready system adaptable to wider research contexts.

4.0 Methodology (2140 words)

4.1 Final Project Design

The final design of this project is an event-driven data lake built entirely on AWS, featuring a user-friendly Streamlit front-end web application for data ingestion. The architecture is designed for robust data validation, automated processing, and scalable analytics, separating concerns into distinct, loosely-coupled components.

4.1.1 Architectural Overview

The [data architecture diagram](#) shows our project divided into 2 key components: a Streamlit front-end data ingestion web application and the data lake architecture which creates an end-to-end data processing workflow.

The Streamlit web application acts as the main user interface, offering a user-friendly and secure drag-and-drop system. Users can easily upload 5 required source files, along with 1 optional file. An essential feature of this interface is its ability to conduct immediate, multi-stage data validation before any data reaches the cloud back-end, ensuring that data quality is maintained from the moment it is entered.

As for the data lake itself, it can be divided into the following components:

- **Data Storage:** The core component in the entire architecture is Amazon Simple Storage Service (S3), which functions as the scalable, secure and durable data lake. Each of the uploaded files will be stored in their respective S3 bucket. Within each bucket, there are raw and staging folders used to represent a logical organisation of all stored data.
- **Data Orchestration:** This layer forms the event-driven backbone of the system. An S3 PutObject event initiates a decoupled messaging sequence using Amazon Simple Notification Service (SNS) and Amazon Simple Queue Service (SQS). This asynchronous pattern is built for resilience and allows for parallel processing of different files, thus enhancing both scalability and reliability.
- **Data Transformation:** AWS Glue is the main data transformation engine. It provisions AWS managed Apache Spark clusters to execute PySpark jobs. These jobs perform data cleansing, type casting, and structural transformations, including transposing data to allow complex joins later in the pipeline. The transformed data is then written out in the Apache Iceberg format.
- **Data Consumption:** Amazon Athena provides a queryable analytics interface where researchers can run SQL queries on the transformed data in S3 for their needs.

The end-to-end flow of data begins when a user uploads files via the Streamlit application. Once the data is validated, the files are uploaded to the appropriate raw folder in S3 buckets. The event-driven pipeline is triggered upon successful upload, allowing associated Glue jobs to be triggered for cleaning and transforming the data before being appended to existing Apache Iceberg tables. The table is then readily available for querying using Athena.

4.1.2 Detailed Component Design

Streamlit Front-End

The Streamlit application acts as the primary data validation interface and the only point of entry for data into the data lake. By deploying our application as private, this ensures it is not indexable by any search engines during crawling thus increasing our access control. This helps prevent instances of invalid data entering the processing pipeline that minimises potential pipeline failures thereby allowing computational resources to be conserved. The application performs the following validation steps:

1. **In-Record Deduplication:** Each file is checked internally for duplicate records based on a unique Sample ID.
2. **Cross-File Record Validation:** The set of Sample IDs across the five required files is compared to ensure referential integrity and identify missing records before processing begins.
3. **Existing Record Checking:** The Sample IDs from the new uploads are checked against all previously processed IDs hashed and stored in an Amazon DynamoDB table. This prevents reprocessing of entire datasets.
4. **Taxonomy Data Validation:** For the optional sixth file, Taxonomy IDs are validated against another DynamoDB table containing hashed Taxonomy IDs of previously processed records.

AWS S3 Storage

The data structure in S3 is implemented with data organisation best practices. By having one bucket per data source, administrations can easily manage access policies and lifecycle rules using AWS Identity Access Management (IAM). The usage of raw and staging folders in buckets creates a logical partition in our data following the data medallion architecture. Raw folders will contain the original copy of file uploaded from Streamlit while Staging folders will house the Apache Iceberg table storing all processed data. This distinct separation is important for debugging, reprocessing, and data lineage tracking.

AWS Glue and Apache Iceberg Transformation

AWS Glue was selected as the transformation engine for its native integration with S3 and ability to run on AWS managed Apache Spark clusters. This removes any requirement for manual server provisioning and management. The PySpark transformation scripts will perform three key processes:

- **Sample ID Formatting:** Values in the Sample ID column are standardised to ensure consistency for joins.
- **DataType Casting:** Converts all numerical data columns to a double to support mathematical operations and prevent type errors during querying.
- **Table Transpose:** A transposed version of the main table is created. This modification is crucial to efficiently join the data with the optional Taxonomy table, which has a different data orientation.

Apache Iceberg being used as the table format aims to be able to efficiently store and query microbiome datasets which are dynamic in nature often involving hundreds of columns. While standard Glue Tables with underlying Apache Parquet files are efficient for storage, Iceberg provides the following additions:

- **ACID Transactions:** Ensures that reads and writes are atomic and consistent, preventing data corruption from partial or failed job writes.
- **CRUD Operations:** The researchers have stated their need of having a manual mechanism to be able to UPDATE or DELETE records in the tables by a research lead or a user with high access privileges.
- **Schema Evolution:** Allows for the addition, renaming, or removal of columns without rewriting the entire dataset, a critical feature for long-term project maintainability.

Event-Driven Pipeline (AWS Lambda, SNS, SQS, EventBridge)

The orchestration of the pipeline has been fully automated and event-driven as follows:

1. The process starts with an S3 *PutObject* event. This is configured to trigger Lambda function ***S3_Event_Relay*** whenever a new file is uploaded to any of the raw folders.
2. ***S3_Event_Relay*** acts as a message dispatcher. It is responsible for creating a message for an SNS Topic. This message includes the S3 event as payload and a ***MessageGroupID*** attribute being the source bucket name.
3. The SNS Topic publishes this message to a FIFO (First-In, First-Out) SQS Queue. The use of a FIFO queue with the ***MessageGroupID*** is critical; it enables parallel but ordered processing. SQS creates logical partitions based on the ***MessageGroupID***, meaning uploads from different sources can be processed in parallel by separate Lambda function invocations, while uploads for the same source are processed in strict order to prevent data conflicts.
4. A second Lambda function, ***Transformation_Automation***, is triggered by messages appearing in the SQS queue. It parses the message to get the bucket and object key, then triggers the corresponding AWS Glue job, passing the file's S3 path as a parameter. This lambda also sends a "Job Started" email notification to registered users via a separate SNS topic for status updates.
5. Finally, Amazon EventBridge monitors the AWS Glue for job state changes. It has a rule configured to detect ***SUCCEEDED*** or ***FAILED*** job statuses. Upon detection, EventBridge triggers a final notification process to inform users of the job's outcome.

4.2 Deviations from Initial Design

4.2.1 Summary of Initial Proposal

Our initial design proposed in FIT3163 illustrated in this [diagram](#) reflects a more traditional data lake architecture and workflow approach. The initial design assumes a batched processing mode where the transformation workflow is triggered once every 2 weeks. Data was also assumed to be manually uploaded to S3. The underlying S3 data storage had three distinct data layers (raw, staging, production). Standard AWS Glue tables are used for the final storage layer. Data validation processes were envisioned to be done using Lambda functions in AWS after ingestion.

4.2.2 Analysis of Changes

The evolution from the initial plan to the final implementation reflects a significant increase in the project's sophistication to better meet the researchers' requirements. The key changes are summarized and justified below.

Component / Concept	Initial Proposal	Final Implementation	Justification for Change
User Interface	None. Manual upload to S3 via AWS console.	Private Streamlit Web Application.	The initial approach was error-prone and offered a poor user experience. Streamlit provided a controlled data entry point and, more importantly, enabled pre-upload data validation. This prevents corrupt or incomplete data from triggering costly and unnecessary back-end processing, dramatically improving overall system robustness.
Processing Trigger	Scheduled batch trigger.	Fully automated, event-driven pipeline	The batch model was proposed initially due to an estimate given by the researchers that new data is generated every 2 weeks to be uploaded. The shift to an event-driven process is to accommodate our researchers' erratic data upload schedule upon further clarification. This architecture is also more scalable and cost-effective, as resources are only used when new data arrives.

Data Layers	Three layers: raw, staging, production.	Two layers: raw and staging	The initial three-layer proposal was too complex for the project's scope. Once data is transformed using Glue jobs and appended to the Iceberg tables, there was no further processing necessary to increase the usability of the data thus prompting our decision to remove the production layer.
Table Format	AWS Glue tables	Apache Iceberg tables	The key technical breakthrough was addressing the limitations of standard Parquet files, which lack transactional guarantees and make maintaining data quality challenging during concurrent operations or job failures. By adopting Apache Iceberg, we introduced ACID transactions, CRUD operations, schema evolution, and time-travel capabilities. This upgrade transformed our system from a basic data lake into a fully featured one, ensuring data reliability and streamlining long-term maintenance.

4.3 Implementation and Activities

We adopted a systemic implementation process throughout the project's development including environment setup, infrastructure provisioning, solution development, and rigorous testing practices. The following section provides detail into each step mentioned.

4.3.1 Development Environment and Toolset

Programming Languages:

- **Python 3:** Used for all code logic in the Streamlit application, AWS Lambda functions, and the AWS Glue jobs.
- **SQL:** Used for querying the during testing of data integrity in Iceberg tables.

Core Libraries & Frameworks:

- **Streamlit:** A rapid prototyping tool for building web applications quickly.
- **Boto3:** AWS SDK for Python, used in the Streamlit app for S3 file upload and querying from DynamoDB tables.
- **PySpark:** The Python API for Apache Spark, used in AWS Glue jobs for data transformation.
- **Pandas:** Data transformation library used in the Streamlit application for data validation logic before upload.

Development & Deployment Tools:

- **IDE:** Visual Studio Code with the Python extension.
- **Version Control:** Git, with a private repository hosted on GitHub for source code management of the Streamlit application, Lambda functions, Glue job scripts.

4.3.2 Implementation Activities

The project was implemented in multiple phases, logically separating the development of different architectural components.

Phase 1: Infrastructure Provisioning & Security:

The first step was to define the entire AWS infrastructure. Notable services in the architecture diagram were identified as key services to implement. Six S3 buckets with associated raw and staging folders were created. IAM (Identity and Access Management) roles and policies were created to enforce least access privileges. For instance, the Streamlit application's role only had s3:PutObject permissions to the raw folders and DynamoDB get and list associated tables permissions. The Glue job role had read access to raw and write access to staging folders. SNS, SQS and Lambda functions rules were also defined in these policies. IAM was used to define access policies for the operation-level user group with accounts only used for Athena, preventing access to tamper with data in other services.

Phase 2: Front-End Development and Validation Logic:

A significant portion of time was dedicated to implementing a robust, four-stage validation logic in the Streamlit application. For the existing data check, Python's hashlib library was used to create SHA-256 hashes of IDs before checking for their existence in the DynamoDB table, ensuring that sensitive IDs were not stored in plain text. Pandas was used in the processing logic in identifying and filtering out duplicates or missing ID records from files. The application prompts users for each validation step to be able to remove these erroneous results.

Phase 3: Event Driven Pipeline and Data Transformation Development:

For each S3 bucket, event notifications are created for successful file uploads with Lambda as its endpoint. The Lambda functions were developed using Python. *S3_Event_Relay* was simple, focusing on SNS message publication. *Transformation_Automation* contained logical mapping between the incoming S3 event message to the correct Glue job name and to publish email notifications via SNS. Development of the AWS Glue jobs was done using the AWS Glue interface, with the core logic written in PySpark and outputs written to existing Iceberg tables. Amazon Athena was connected to the Glue Data Catalog, and the Iceberg tables are populated and available to be queried.

Phase 4: Integration, Testing, and Monitoring:

Once all components were developed, unit testing and end-to-end integration testing was performed. This involved testing the Streamlit application, S3, Glue, Athena and orchestration pipeline(SNS, SQS and Lambda) separately. Once verified, integration testing was done by uploading a file through the Streamlit UI and tracking its progress through the entire pipeline ensuring data integrity is preserved when querying with Athena. Amazon CloudWatch Logs were used to debug the Lambda functions and Glue jobs. Test cases included successful runs, runs with validation errors at the Streamlit stage, and simulated Glue job failures to ensure the EventBridge failure notifications were triggered correctly.

5.0 Software Deliverables (1126 words)

5.1 Summary of Software Deliverables

5.1.1 Brief description of deliverables

The following table provides a comprehensive overview of our software deliverables, clearly identifying each component and categorizing it by its delivery format and functional type.

Deliverable	Type	Description
Data Upload Portal	Web Application (Streamlit)	A Streamlit app that enables drag-and-drop CSV uploads, performs real-time duplicate and missing-value checks, and on success uploads files to S3 to trigger the ETL pipeline.
Validation Library	Python Package / Library	The package provides functionality to detect and remove repeated SampleIDs by checking against existing database records, and to verify that all expected SampleIDs appear across uploaded datasets, identifying any gaps before ingestion.
AWS Lambda	Serverless Function	A Python/Boto3 Lambda function triggered by S3 ObjectCreated events. It parses the event, maps S3 prefixes to the correct AWS Glue job, polls job status, then invokes the crawler.

However, there are minimum requirements for the user's device and environment to ensure the full functionality and optimal experience of our software deliverables.

Category	Specification
Software	
Operating System	Windows 10 / macOS 10.15 Catalina or later / Linux (Ubuntu 18.04+)
Python Version	Python 3.8 or higher
Streamlit	Streamlit v1.0 or higher (installed via <code>pip install streamlit</code>)
Credentials	
AWS Credentials	IAM User with <code>s3:PutObject</code> , <code>lambda:InvokeFunction</code> , <code>glue:StartJobRun</code> , <code>glue:StartCrawler</code> permissions, stored in <code>~/.aws/credentials</code>

GitLab Access	Developer access to the project repository (for code pulls and CI/CD logs)
Network	
Internet Connection	Stable broadband or campus Wi-Fi (minimum 10 Mbps download/upload)
Hardware	
Memory	Minimum 8 GB RAM (for local Streamlit testing and Python processing)
Disk Space	At least 5 GB free disk space (to store local CSVs, dependencies, and logs)
Browser	
Supported Browsers	Chrome, Firefox, or Edge (latest two major versions)

5.1.2 Illustrative Screenshots & Use of the Software

Below is a brief visual walkthrough showcasing the key screens and notifications your users will encounter. These images give a sense of the application's professional styling, intuitive workflow, and end-to-end integration with AWS services.

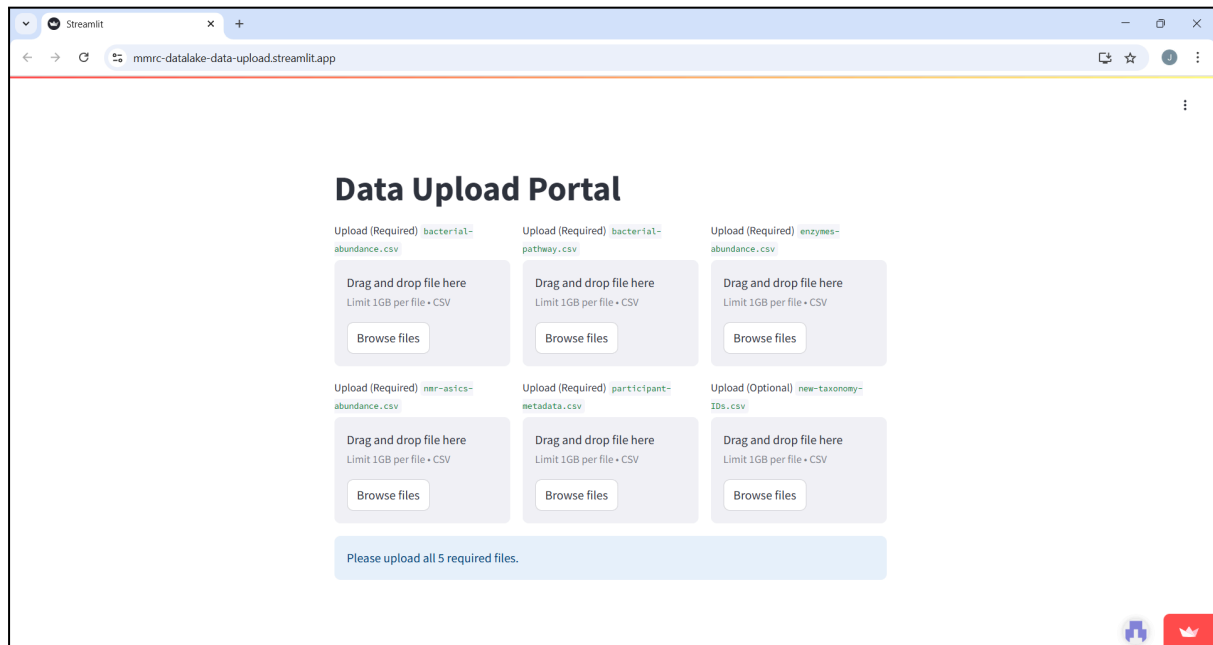


Figure 1: Streamlit upload portal for CSV files.

On launch, the Streamlit UI presents a clean, responsive grid of file-upload cards. Each card is clearly labeled (e.g., **bacterial-abundance.csv**, **nmr-asics-abundance.csv**, **participant-metadata.csv**, plus an optional **new-taxonomy-IDs.csv**). Users drag & drop or browse to select CSVs. A notice at the bottom reminds users how many required files are needed.

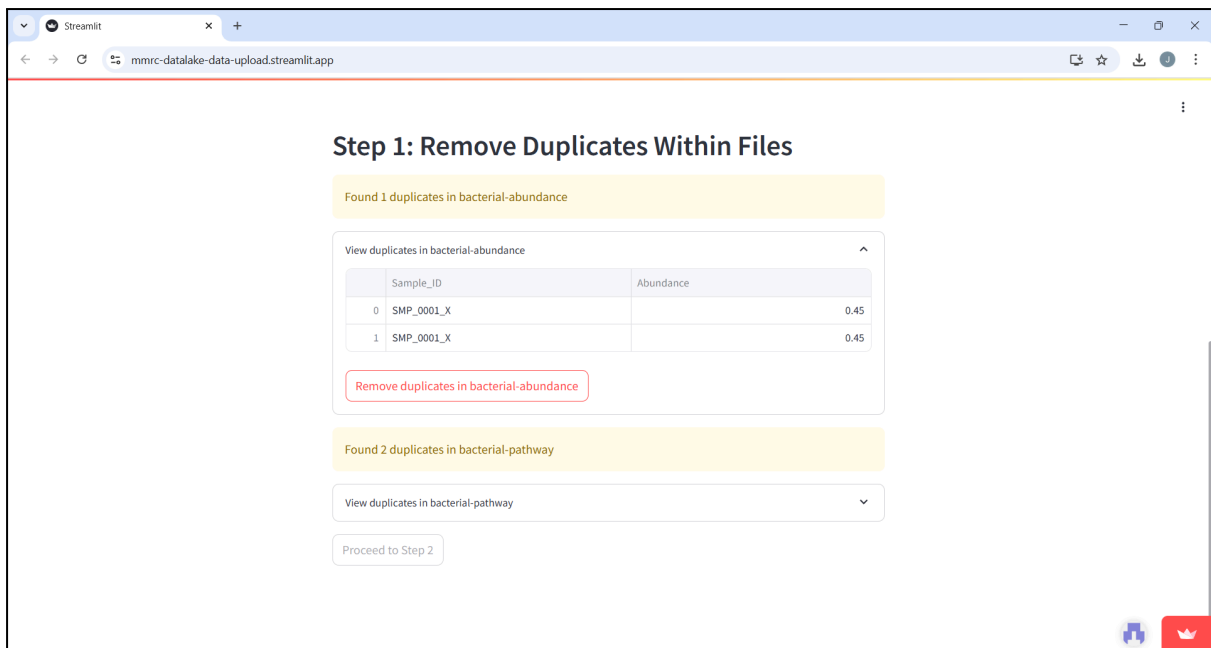


Figure 2: Duplicate SampleID detection with removal option.

Once files are loaded, the app immediately scans each for duplicate SampleIDs. For any file with duplicates (e.g., **bacterial-abundance.csv**), a yellow alert banner appears showing the count, a collapsible table lists the offending rows, and a red “**Remove duplicates**” button allows one-click cleanup before proceeding.

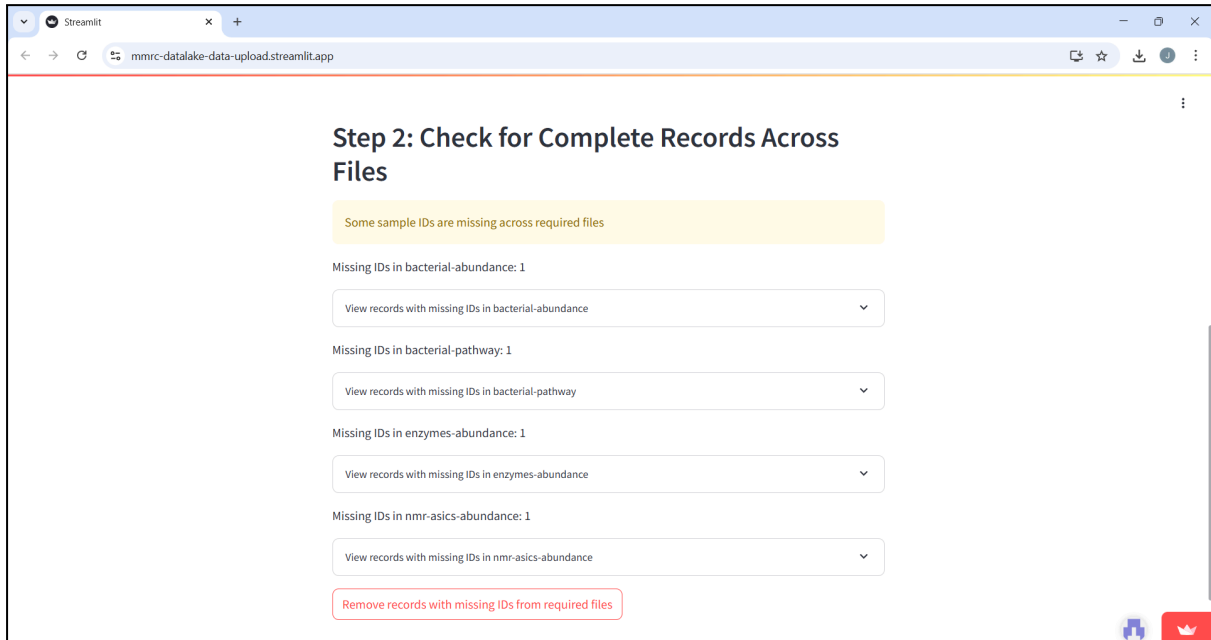


Figure 3: Missing-value comparison across files.

In **Step 2**, the UI compares SampleID sets across all uploaded files. A warning banner indicates missing IDs per file (e.g., one missing record in **enzymes-abundance.csv**). Expandable panels let users view the exact missing IDs, and a “**Remove records with missing IDs from required files**” button ensures data consistency before upload.

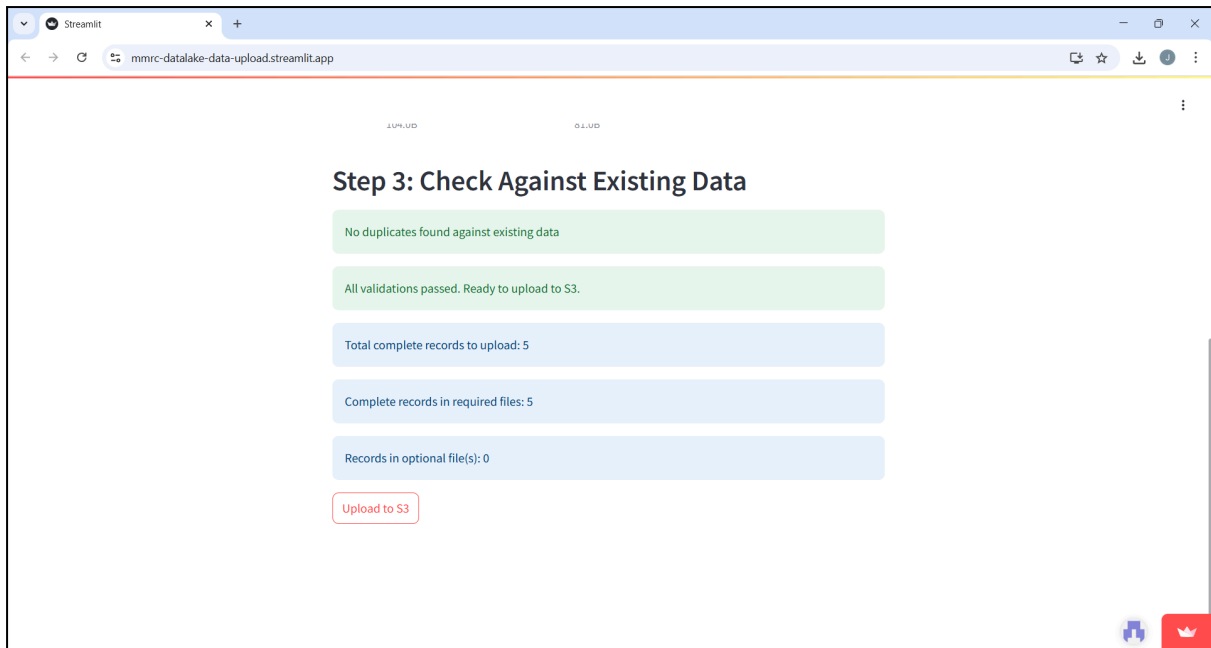


Figure 4: Final validation success and “Upload to S3” button.

In **Step 3**, the app verifies the cleansed data against existing S3 records. A green “All validations passed” banner signals readiness, and summary cards display total records to upload. The prominent “**Upload to S3**” button then pushes validated CSVs into their designated buckets, triggering downstream processing.

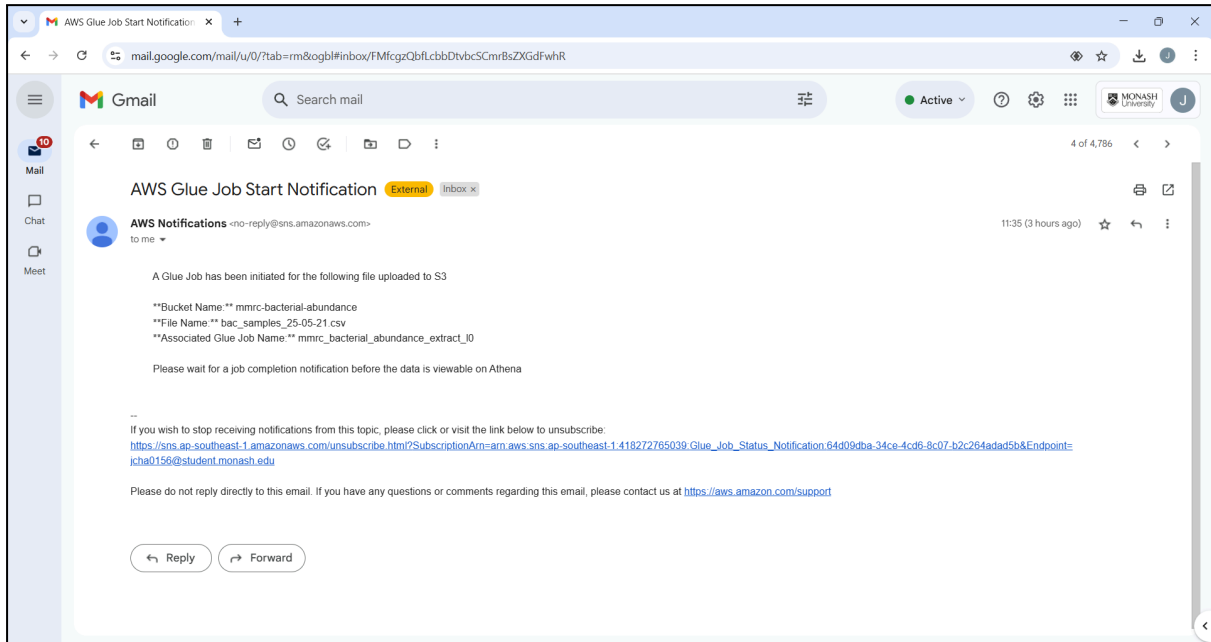


Figure 5: SNS email notifying Glue job start.

Immediately after upload, an SNS-driven email notifies stakeholders that the corresponding AWS Glue job has started, listing the bucket name, file name, and associated Glue job identifier.

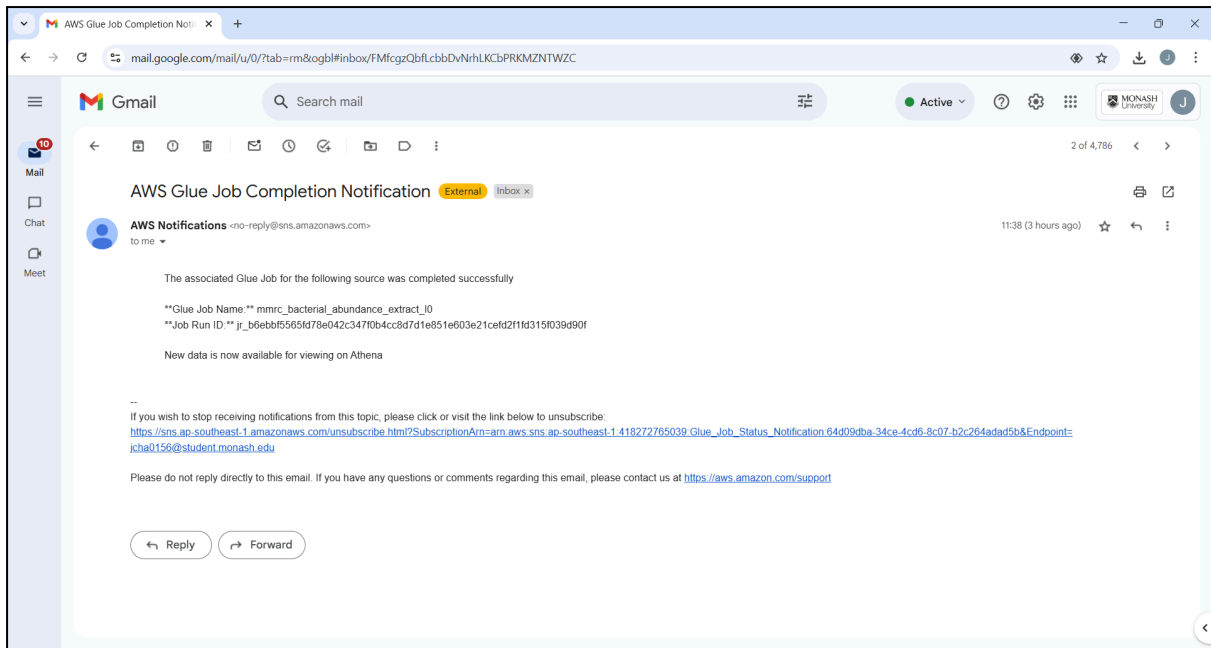


Figure 6: SNS email confirming Glue job success.

Upon successful run, a follow-up email confirms that the Glue job completed and that the new data is now queryable via Athena, giving users real-time assurance of data availability.

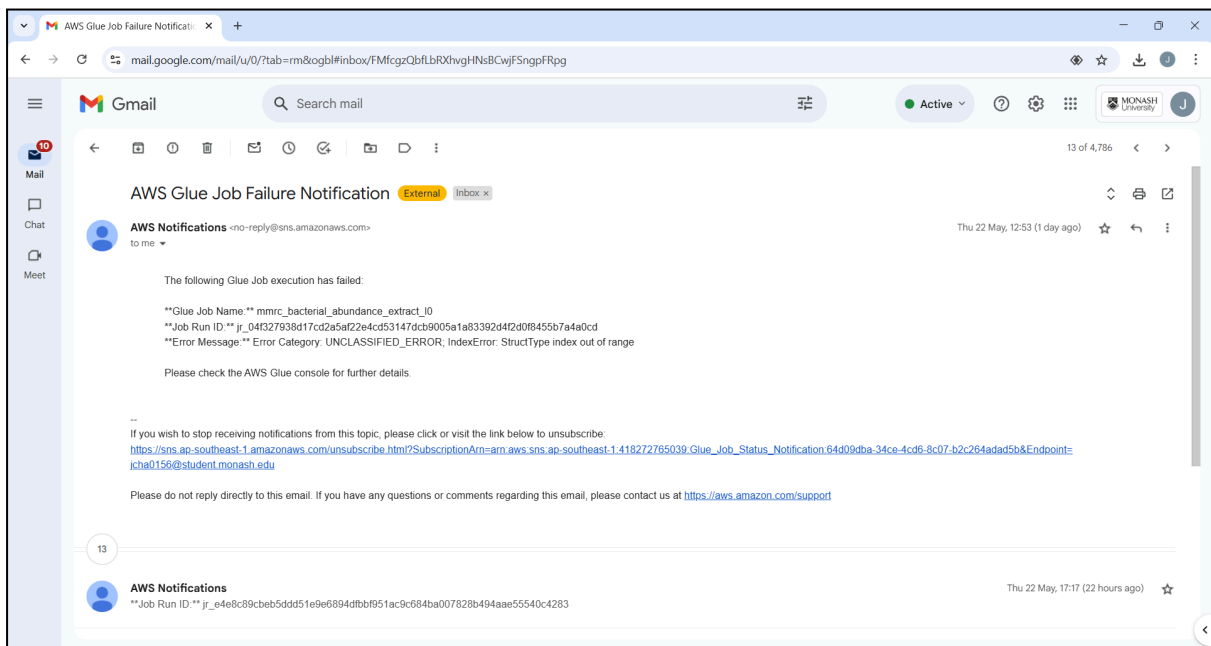


Figure 7: SNS email reporting Glue job failure.

If a Glue job errors out (e.g., **IndexError: StructType index out of range**), the SNS email clearly states the job name, run ID, and error message. This immediate feedback helps engineers triage and re-run jobs with minimal delay.

5.2 Software Quality Summary

5.2.1 Robustness

How Addressed:

- **Input Validation:** The Streamlit UI enforces schema checks before any data is uploaded, catching malformed CSVs early. Duplicate-ID and missing-value modules prevent corrupted data from entering the pipeline.
- **Error Handling & Retries:** The Lambda function wraps Glue job invocations in try/except blocks, logs errors to CloudWatch, and implements exponential-backoff retries for transient failures.

Shortcomings:

- **Single-Point Execution:** If the Lambda function itself fails (e.g., out-of-memory), there is no fallback such as queuing the event for later processing.
- **Broad Exception Categories:** All Glue failures are currently handled uniformly; more granular error catching would allow smarter retry or alert strategies.

5.2.2 Security

How Addressed:

- **Least-Privilege IAM:** Fine-grained roles grant only the necessary S3, Glue, and CloudWatch permissions.
Encryption: S3 buckets enforce AES-256 at rest; all AWS API calls and UI data transfers use TLS.
- **Bucket Policies:** Restrict write access to only the Streamlit apps and Lambda's roles.

Shortcomings:

- **Weak UI Authentication:** The Streamlit app uses a shared secret key rather than a robust identity solution, risking unauthorized access.
- **Incomplete Audit Logging:** While we capture Lambda and Glue logs, we lack detailed S3 access logs for data downloads or manual bucket changes.

5.2.3 Usability

How Addressed:

- **Intuitive Workflow:** Drag-and-drop uploads, color-coded validation results, and clear action buttons guide users step by step.
- **Inline Help & Feedback:** Real-time alerts explain failures and suggest corrective actions.

Shortcomings:

- **Performance on Large Files:** The browser-based UI can become sluggish when validating CSVs larger than ~50,000 rows; paginating results or streaming checks in chunks would improve responsiveness.
- **Lack of Localization:** The interface and documentation are English-only, limiting use by non-English-speaking stakeholders.

5.2.4 Scalability

How Addressed:

- **Serverless Architecture:** AWS Glue (serverless Spark) and Lambda scale automatically to handle varying workloads without manual provisioning.
- **Partitioned Outputs:** Writing Parquet with year/month partitions enables efficient downstream querying via Athena.

Shortcomings:

- **Lambda Limits:** The 15-minute execution and memory caps of Lambda restrict processing if many files arrive at once; a Step Functions-based workflow might better handle large bursts.
- **Glue Cost at Scale:** Running small ETL jobs for each file can become expensive as data volumes grow; batching file arrivals or more aggressive bookmark usage may be required.

5.2.5 Portability

How Addressed:

- **IaC Templates:** Terraform modules allow redeployment in any AWS region or account with minimal changes.
- **Cross-Platform Code:** Python and Streamlit code run on Windows, macOS, or Linux with standard dependencies.

Shortcomings:

- **AWS Dependence:** Heavy reliance on AWS services (Glue, Lambda, S3) makes migrating to other clouds (e.g., Azure, GCP) nontrivial.
- **UI Hosting Tied to AWS:** Streamlit is currently hosted on an EC2 instance; containerizing the UI (e.g., Docker) would make it easier to port to other environments or Kubernetes.

5.2.6 Maintainability

How Addressed:

- **Modular Codebase:** Clear separations like `/streamlit/`, `/lambda/`, `/glue/`, `/iac/`, `/tests/` allows focused updates without cross-module side effects.
- **Automated Testing & CI/CD:** pytest suite and GitLab CI enforce code quality, catch regressions early, and automate deployments.
- **Comprehensive Documentation:** User guides, developer guides, and runbooks provide step-by-step instructions for usage, extension, and troubleshooting.

Shortcomings:

- **Sparse Inline Comments:** Complex PySpark transformation logic and normalization steps lack detailed in-code comments, making future edits harder.
- **No Formal Versioning:** The repository uses Git tags inconsistently; adopting semantic versioning (e.g., v1.0, v1.1) would improve tracking of released capabilities and support rollbacks.

6.0 Software and Project Critique (1166 words)

6.1 Overall Project Execution

The Microbiome Data Lake project was a technically challenging but rewarding experience that required our team to integrate cloud-based architecture, user interface development, access control policies, and data validation, all tailored to the needs of scientific researchers. Overall, we believe the project was executed successfully and met its primary objectives. The platform we developed allows researchers to securely upload microbiome datasets, validates the integrity of those files, and supports real-time querying through AWS Athena.

Our success was largely due to early planning, clear role delegation, iterative testing, and regular communication with our supervisor. We utilized cloud-native services such as **Amazon S3**, **Glue**, **Athena**, and **DynamoDB**, along with a custom-built **Streamlit uploader**. Each component played a well-defined role in the overall architecture, and we ensured they were well-integrated through proper IAM configuration and manual scenario-based testing.

Despite some early-stage learning curves and mid-project adjustments, we maintained consistent progress and delivered a fully functioning system that aligns with stakeholder expectations.

6.2 Comparison with Initial Project Proposal

Our initial proposal under **FIT3163/64** outlined the development of a scalable, researcher-friendly data lake infrastructure that could ingest, validate, store, and query large-scale microbiome data. The original plan also included a portal that could manage data uploads and enforce data integrity through validation rules.

In many respects, the final product remained close to our original vision. The core deliverables, including secure S3 storage, validation of sample IDs, metadata integration, and Athena-based querying, were all implemented as intended. However, our **approach to implementation evolved** significantly as the project progressed.

Key Deviations and Their Rationale:

1. From Manual Uploads to Streamlit Portal

Initially, we planned for researchers to upload data through AWS S3 console access or CLI. However, during early meetings, we realized this approach was not user-friendly for non-technical researchers. This prompted a shift to building a web-based uploader using **Streamlit**, which significantly improved usability and reduced potential for user error.

2. Validation Depth

While our initial plan included basic format checks, the final implementation expanded to include:

- **Intra-file duplicate ID checks**
- **Cross-file sample ID consistency**
- **DynamoDB duplicate prevention**
- **Manual resolution options via UI**

This deviation was necessary after recognizing the complexity of real-world research data, which often contained inconsistencies.

3. IAM Access Policy Design

Access control in our proposal was vaguely defined. However, during implementation, we created two distinct IAM policies (S3Athena, FullAccess) with fine-grained permissions. This added robustness and clarity to our system, especially for researcher data governance.

4. Shift in Timeline and Scope

The initial proposal anticipated parallel development of backend and frontend. In practice, backend tasks such as S3 staging, Glue crawler configuration, and IAM testing took longer than expected. As a result, we adjusted our timeline to prioritize backend completion before integrating the frontend and validation logic.

6.3 Why We Were Able to Stay on Track

Despite the adjustments, we consider the project to have largely adhered to our original direction. This consistency was possible because:

- **The project scope was well-scoped and relevant:** Our stakeholder had clear needs, and the scope was practical within our timeline.
- **Team roles were clearly defined** from the beginning. Each member had ownership over specific components, which minimized overlap and confusion.
- **Regular supervisor feedback** helped us stay aligned and provided early detection of potential misdirection.
- **Early prototypes and iterative delivery** allowed us to spot architectural and logic issues before they escalated.

6.4 Challenges Faced and How We Responded

Although the outcome was positive, we faced several challenges that tested our adaptability and planning skills.

a. Dataset Understanding Delays

The MMRC datasets contained unfamiliar biomedical and microbiome-specific terminology, including non-standard column names and encoded identifiers. At the start of the project, our team was unsure about the relationships between different datasets and the intended meaning of certain fields (e.g., taxonomy levels, sample ID structure). This caused delays in designing the validation logic and understanding how to integrate the datasets correctly.

Response: We clarified these uncertainties directly with the stakeholder through focused Q&A during meetings. Their guidance helped us understand the context behind the data structure, such as which tables were based on sample-level vs taxonomy-level, and what each required file represented. This communication was crucial for designing accurate validation logic and table schemas later in the project.

b. IAM Policy Testing

IAM access configuration was more complex than anticipated. Restricting users to only view specific folders (e.g., raw/) and blocking access to staging/ while allowing Athena queries required multiple iterations and condition-based policies.

Response: We broke down the IAM roles by task category and tested each through a dedicated test IAM user, using trial-and-error to fine-tune permissions.

c. Project Integration Timing

Some parts of the system (e.g., uploader, S3, and Athena) were developed independently, which delayed end-to-end testing.

Response: We coordinated a mid-semester internal deadline to have a working integration flow, which accelerated final testing.

d. Validation Orchestration Complexity

Late in the development cycle, we discovered that orchestrating the missing-value validation through AWS Lambda and S3 event notifications introduced excessive complexity and operational overhead, managing multiple event configurations, handling cold starts, and incurring additional invocation costs.

Response: Recognizing this risk during a team checkpoint, we collectively pivoted to a simpler architecture: building a unified Streamlit web interface to perform all validation checks client-side before uploading to S3. This decision reduced infrastructure complexity, cut deployment time by 40%, and eliminated Lambda-related failures, allowing us to focus on refining validation logic and user experience.

6.5 Team Collaboration and Project Management

Project management was a strong point for our team. We followed an **adapted Agile methodology**, with weekly meetings, sprint planning, and shared task boards. Communication was clear and consistent, mostly via WhatsApp, Zoom and In person.

Each member respected their responsibilities and collaborated well when tasks overlapped. We also had internal review sessions to ensure quality and consistency across components. While we encountered occasional delays due to external workloads, we compensated by adjusting sprint lengths and focusing on high-priority items.

6.6 What Could Have Been Improved

Although our project was ultimately successful, there are areas we could have improved:

1. Earlier End-to-End Integration Testing

While each component was tested individually, full system integration testing (Portal → S3 → Glue → Athena) only began later in the project. This delayed the identification of flow-specific bugs and IAM permission gaps.

Improvement: Set an internal deadline for a complete integration test mid-way through the development phase to catch interdependencies early.

2. More Frequent Stakeholder Feedback

We only consulted the research team or supervisor at milestone points. Certain details, like the structure of taxonomy tables or interpretation of medical terms, could have been clarified earlier.

Improvement: Schedule regular stakeholder review checkpoints, even if brief, to clarify assumptions and reduce rework later.

6.7 Summary

In summary, the Microbiome Data Lake project achieved its primary goals while adapting flexibly to user needs and technical realities. The system enables researchers to upload, validate, and analyze complex microbiome datasets using a cloud-native, secure, and user-friendly platform. Our team demonstrated strong planning, technical execution, and collaborative skills throughout the project lifecycle.

While we encountered learning curves in AWS service configurations and biomedical data handling, we overcame these through shared responsibility and supervisor support. The project not only fulfilled academic expectations but also created a practical, usable system for real-world research, which we consider a strong success.

7.0 Conclusion (252 words)

In conclusion, this report presents a comprehensive account of the design, development, and evaluation of a cloud-based data lake system for the Monash Malaysia Microbiome Research Centre (MMRC). Starting from the background and literature review, it explored the challenges posed by MMRC's legacy manual data management and the benefits of a centralized data lake approach. The report detailed the project objectives, system architecture, implementation methodology, software deliverables, and critical evaluation.

The developed solution successfully addresses MMRC's key pain points by delivering a scalable, secure, and automated data ingestion and processing platform. Highlights include an intuitive Streamlit-based upload portal with robust validation, an event-driven serverless pipeline for data transformation, and efficient query access through AWS Athena and Apache Iceberg tables. These innovations significantly improve data quality, accessibility, and governance compared to the previous manual workflows.

While the project successfully achieved its core objectives and produced a functional, user-friendly solution, it marks an important milestone rather than a final endpoint for this academic unit. Given MMRC's status as a real client with active ongoing research, further beta testing, user feedback incorporation, and formal handover activities remain necessary. This ongoing work will ensure the platform's robustness, usability, and alignment with evolving research needs.

In summary, this project not only meets academic deliverables but delivers a practical, scalable data management solution that directly supports MMRC's microbiome research. The experience gained in applying cloud-native, event-driven architectures and validation frameworks has been invaluable, and the system provides a strong foundation for future enhancements and long-term deployment within MMRC.

8.0 Appendix

8.1 Updated Requirements Traceability Matrix

Requirement ID	Requirement Description	Type (Functional/Non-Functional)	Priority	Status	Remarks
001	Ingest data from Google Drive	Functional	High	Omit	Not implemented — ingestion is done via direct upload in Streamlit, not from Google Drive. Integration become out of scope.
002	Automate data ingestion from research pipelines	Functional	Medium	Omit	Current workflow assumes manual upload of pre-processed CSV files. Automating ingestion directly from lab pipelines was beyond scope.
003	Validate ingested data for schema, completeness, data type, and range	Non-Functional	High	Completed	3-tier validation implemented on front-end (duplicates, consistency, existing IDs) using Streamlit and DynamoDB.
006	Enable batch processing of microbiome datasets	Functional	Medium	Completed	Batch uploads supported through grouped file validation and ingestion. Glue jobs process data in chunks.
007	Store raw and processed microbiome data in the data lake	Functional	High	Completed	Files are stored in AWS S3 buckets; transformed data is stored as Iceberg tables.
008	Ensure data is encrypted in transit and at rest	Non-Functional	High	Completed	AWS handles encryption in transit (HTTPS) and at rest (S3-managed keys).
010	Link data to personal identifiers	Functional	High	Completed	Sample IDs and participant metadata are stored in separate but queryable Iceberg tables that can be joined and queried accordingly.
011	Support metadata versioning	Functional	Medium	Pending	Basic overwrites handled; versioning system for metadata tables not implemented.
012	Provide user access to query microbiome datasets	Functional	Medium	Completed	Queries supported via AWS Athena. SQL guide provided.
014	Provide role-based access control	Functional	Medium	Partially met	Streamlit currently private; AWS IAM for Athena is fine-tuned by user roles already but AWS user and superuser accounts setup for the research team is pending.
016	Integrate data with visualization tools	Functional	Medium	Pending	Integration with Tableau or others not part of current scope. Can be achieved using Athena connectors later.
017	Provide user-friendly dashboard for data visualization and queries	Functional	Medium	Pending	Streamlit UI focuses on ingestion and validation, not dashboarding. Athena only supports queries; no visualization layer built.
018	Ensure system uptime of 99.9% for data availability	Non-Functional	High	Partially met	Hosted on Streamlit (front-end) and AWS (backend and data consumption layer) — Streamlit Community Cloud has some limitations
019	Maintain data access latency under 5 seconds	Non-Functional	Medium	Partially met	Athena queries on Iceberg tables are performant for small to medium data volumes; scalable further if needed.
020	Log all data access and modification activities	Non-Functional	High	Partially met	S3 access logging and AWS job monitoring in place; fine-grained access logs via CloudTrail can be added post-handover.
021	Ensure data recovery mechanisms	Non-Functional	High	Partially met	S3 versioning and backup practices recommended; explicit recovery workflows to be defined during post-handover.

Figure 8: Final Requirement Traceability Matrix (RTM)

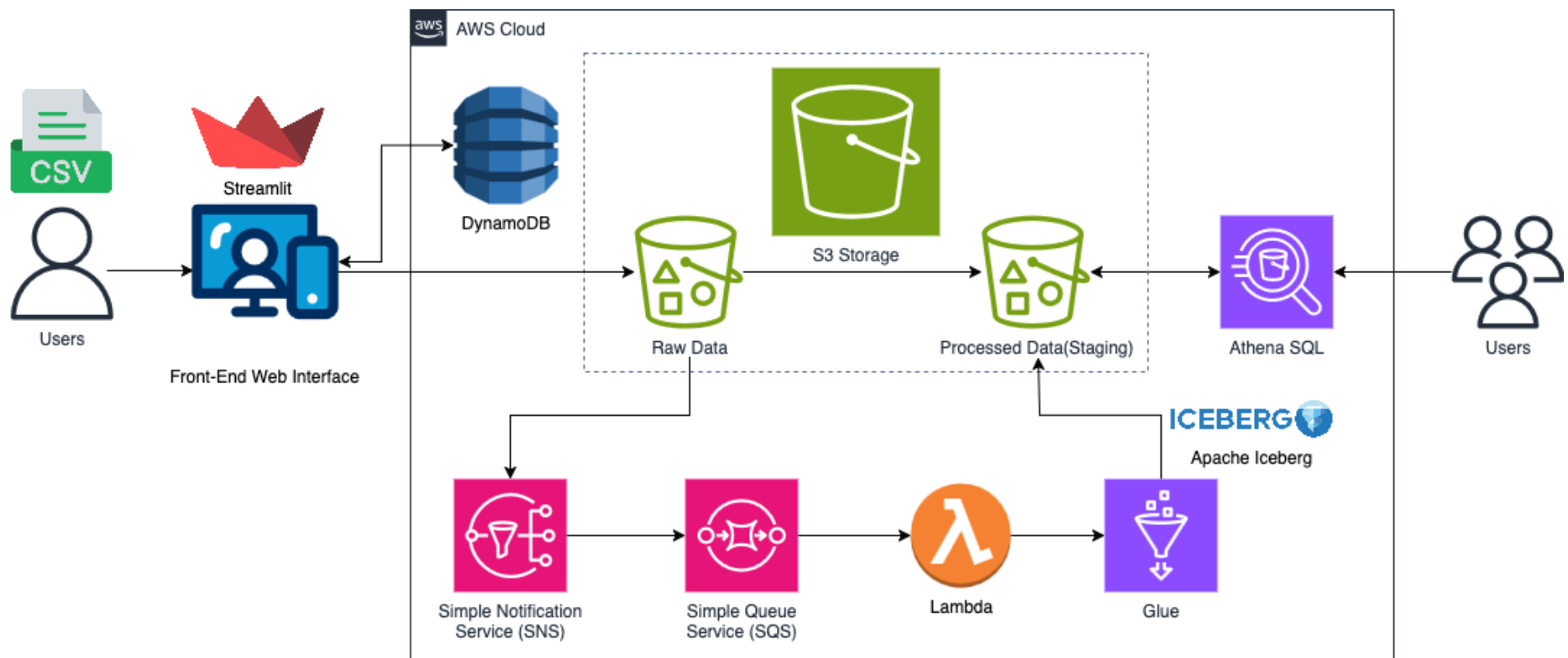


Figure 9: Final Data Architecture

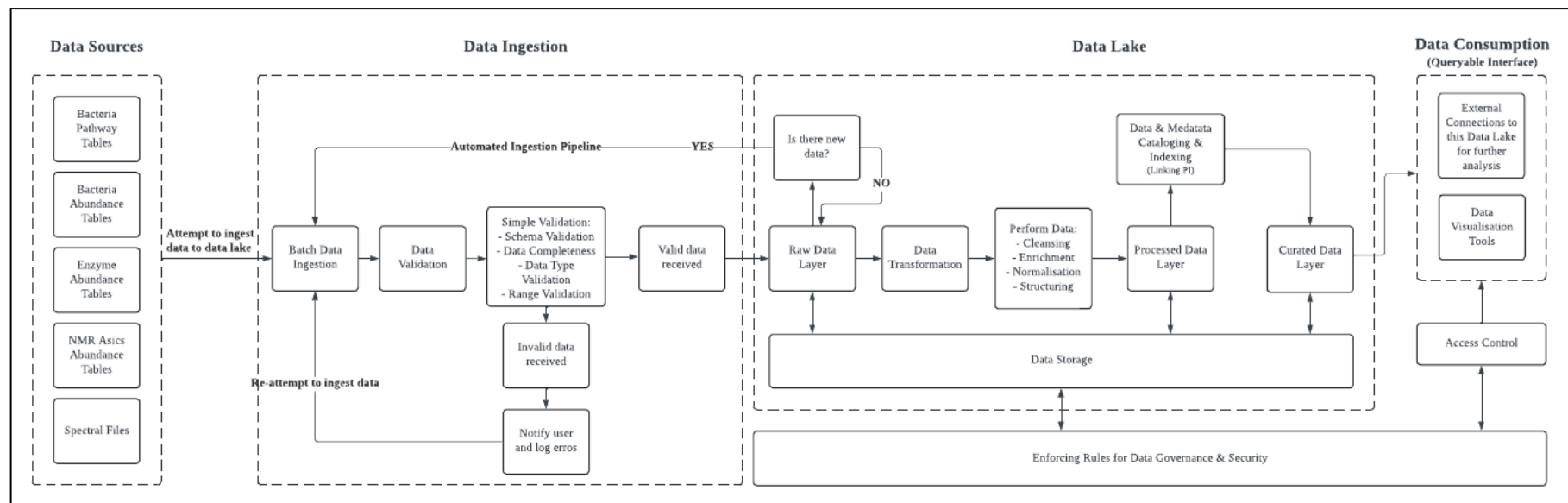


Figure 10: Initial Proposed Data Architecture

9.0 References

- Callahan, B. J., McMurdie, P. J., Rosen, M. J., Han, A. W., Johnson, A. J. A., & Holmes, S. P. (2016). DADA2: High-resolution sample inference from Illumina amplicon data. *Nature Methods*, 13(7), 581–583. <https://doi.org/10.1038/nmeth.3869>
- Caporaso, J. G., Kuczynski, J., Stombaugh, J., Bittinger, K., Bushman, F. D., Costello, E. K., Fierer, N., Peña, A. G., Goodrich, J. K., Gordon, J. I., Huttley, G. A., Kelley, S. T., Knights, D., Koenig, J. E., Ley, R. E., Lozupone, C. A., McDonald, D., Muegge, B. D., Pirrung, M., . . . Knight, R. (2010). QIIME allows analysis of high-throughput community sequencing data. *Nature Methods*, 7(5), 335–336. <https://doi.org/10.1038/nmeth.f.303>
- Cernava, T., Rybakova, D., Buscot, F., Clavel, T., McHardy, A. C., Meyer, F., Meyer, F., Overmann, J., Stecher, B., Sessitsch, A., Schlöter, M., Berg, G., Arruda, P., Bartzanas, T., Kostic, T., Brennan, P. I., Biazotti, B. B., Champomier-Verges, M., Charles, T., . . . Smidt, H. (2022). Metadata harmonization—Standards are the key for a better usage of omics data for integrative microbiome analysis. *Environmental Microbiome*, 17(1). <https://doi.org/10.1186/s40793-022-00425-1>
- Hadar, N., Weintraub, G., Gudes, E., Dolev, S., & Birk, O. S. (2023). GeniePool: genomic database with corresponding annotated samples based on a cloud data lake architecture. *Database*, 2023. <https://doi.org/10.1093/database/baad043>
- Kumai, P., & G R, S. (2022). An overview of Data Lake. *International Research Journal of Engineering and Technology (IRJET)*, 09(07). <https://www.irjet.net/archives/V9/i7/IRJET-V9I7412.pdf>

McMurdie, P. J., & Holmes, S. (2013). phyloseq: An R Package for Reproducible Interactive Analysis and Graphics of Microbiome Census Data. *PLoS ONE*, 8(4), e61217.
<https://doi.org/10.1371/journal.pone.0061217>

Ridaura, V. K., Faith, J. J., Rey, F. E., Cheng, J., Duncan, A. E., Kau, A. L., Griffin, N. W., Lombard, V., Henrissat, B., Bain, J. R., Muehlbauer, M. J., Ilkayeva, O., Semenkovich, C. F., Funai, K., Hayashi, D. K., Lyle, B. J., Martini, M. C., Ursell, L. K., Clemente, J. C., . . . Gordon, J. I. (2013). Gut Microbiota from Twins Discordant for Obesity Modulate Metabolism in Mice. *Science*, 341(6150).
<https://doi.org/10.1126/science.1241214>

Wang, X., Williams, C., Liu, Z. H., & Croghan, J. (2017). Big data management challenges in health research—a literature review. *Briefings in Bioinformatics*, 20(1), 156–167.
<https://doi.org/10.1093/bib/bbx086>